APPLYING F#, IMPROVING F#, DELIVERING F#

Don Syme, Phillip Carter

SUMMARY

It's been a great 12 months!

My preferred way of working is to iterate:

1. Apply F# in an important area
2. Learn and digest
3. Improve F# in incremental but important ways

Please do likewise!

PART I - APPLYING F# TO MOBILE

Part I

fsprojects.github.io/Fabulous

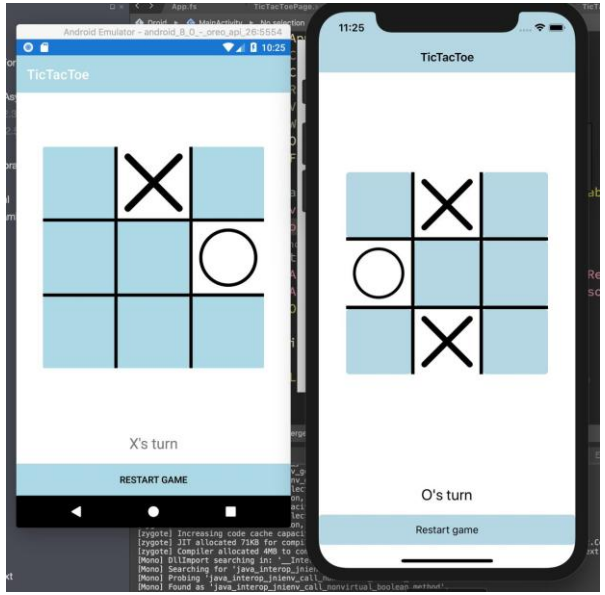
THANKS

- Timothé Larivière
  - co-maintainer on Fabulous, author of ElmishContacts and ElmishPlanets demos
- Jim Bennett
  - will talk a bit about using Fabulous and give a demo
- Frank Krueger
  - ImmutableUI was inspiration and initial partial list of Xamarin.Core bindings
- Boris D. (github @dboris)
  - authored "Elmish.Forms" ("half-elmish" for static views only)
- Jason Imison
  - F# in Visual Studio for Mac and Xamarin
- Eugene Tolmachev
  - Creator or "Elmish" for Fable, used for WPF and others
- Justin Sacks and others
  - Creators of "Elmish.WPF" ("half-elmish" for static views only)
- Alfonso, Maxime, Steffen and many others
  - The Fable gang



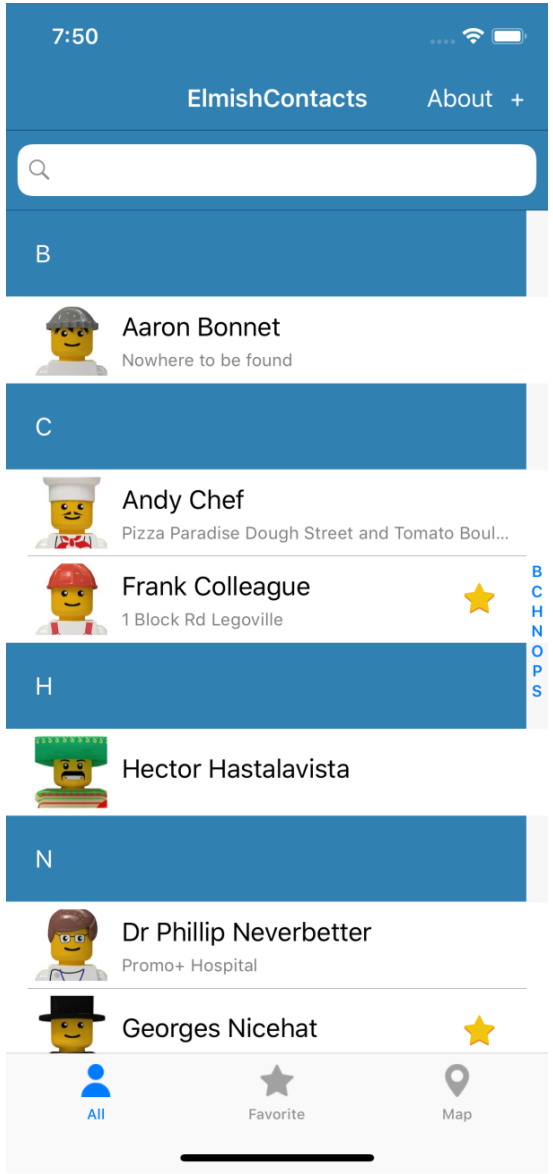


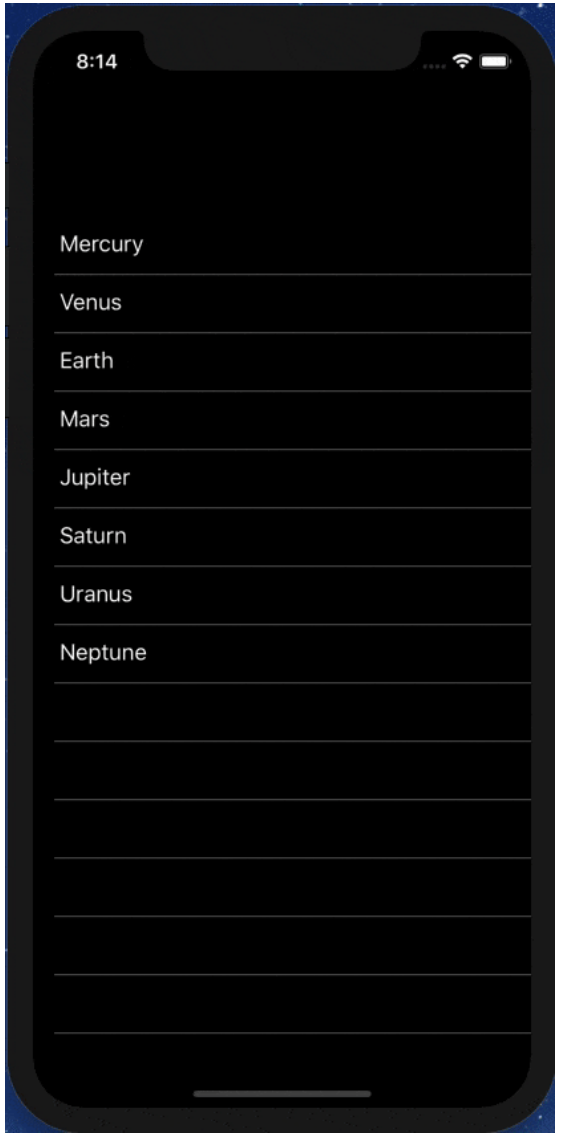


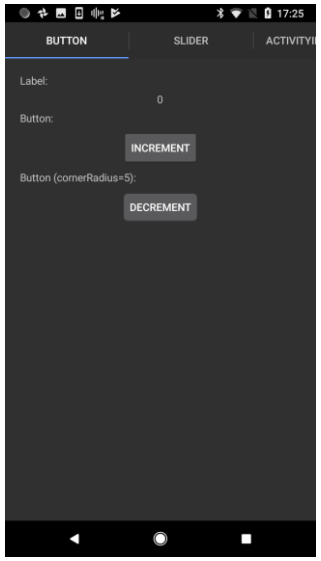


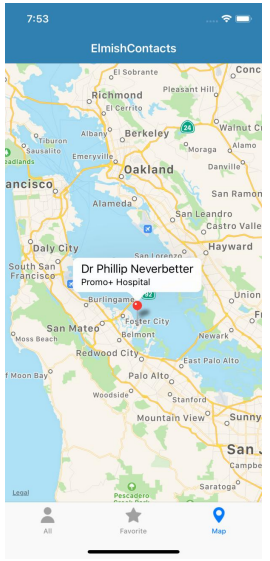
GET IT ON Google Play

Download on the App Store









THE CONTEXT

- Early F# embraced functional programming but lacked opinion on functional app development
- Fable solves this for Web Development
- F# developers also need to be able to go mobile.
- One question in 2018: Can we make Xamarin a **simple** and **compelling** for F# app development?

THE PROBLEM WITH XAML

- A core problem: Xaml
  - Xaml assumes "designers" will "sliced off" UI from code into a separate DSL.
  - Xaml is **static**, with **layers of complexity** (e.g. templating), and **code-like constructs** (behaviours, convertors), and requires **complex designer tooling** (which often fails).
  - Separating the UI is fine, but do we really need a separate language to do it?

THE PROBLEM WITH XAML

how to "!" in a binding in xaml

GeorgeCook

March 2015 edited March 2015 in Xamarin.Forms

i.e.

`

```
<Button Clicked="OnLoginClicked"
    Image="Images/Login/loginWithFacebookButton.png"
    IsEnabled="{!(Binding IsLoggingIn)}"  <<<<<<< this is the bit that I don't know how to do.
```

THE PROBLEM WITH XAML

JoeManke

March 2015

What you want is a converter.

```
<local:NegateBooleanConverter x:Key="inverter"/>
<Button Clicked="OnLoginClicked"
        Image="Images/Login/loginWithFacebookButton.png"
        IsEnabled="{Binding IsLoggingIn, Converter={StaticResource inverter}"/>
```

Then in your code:

```
    public class NegateBooleanConverter : IValueConverter
    {
        public object Convert (object value, Type targetType, object parameter, System.Globalization.CultureInfo cultu
        {
            return !(bool)value;
        }
        public object ConvertBack (object value, Type targetType, object parameter, System.Globalization.CultureInfo c
        {
            return !(bool)value;
        }
```

Good answer but someone who is not much familiar with Xaml they need to add the name space in xaml file also to use that converter

INSPIRATION

- Fable has popularized the "Elm" architecture in F#
  - Fable+Elmish makes web app development simple
  - All F# code
  - View descriptions are separated but simple, and in F#

- I'm very influenced by SAFE Stack
  - Non-F# people have found Fable+Elmish easy in business contexts

http://fable.io http://github.com/elmish

https://safe-stack.github.io/

THE NEED

What's needed?

the simplicity of Fable + Elmish, but for Xamarin

(aside: you can do Fable+Elmish to ReactNative already)

A SIMPLE FABULOUS APP

```
/// The model from which the view is generated
type Model =
    { Pressed: bool }

/// The messages which cause updates to the model
type Msg =
    | Pressed

/// Returns the initial state
let init() =
    { Pressed=false }

/// The function to update the
let update (msg: Msg) (model:
    match msg with
    | Pressed -> { model with

 /// The function to produce t
let view (model: Model) dispat
    if model.Pressed then
        View.Label(text="I was
    else
        View.Button(text="Pres
```

```
type App () as app =
    inherit Application ()

    let runner =
        Program.mkProgram App.init App.update App.view
        |> Program.runWithDynamicView app
```

GETTING STARTED

1. Install Visual Studio

   (Mac or Windows with .NET Core and Xamarin enabled)

2. Create an app

```
dotnet new -i Fabulous.Templates

dotnet new fabulous-app -lang F# -n FriedChickenApp
```

3. Plug in your Android via USB, enable developer mode, install USB drivers (Windows) etc.

4. Build/deploy

```
msbuild FriedChickenApp\FriedChickenApp.sln
```

THE VIEW DSL

1. Views are expressed using a DSL
2. It's like Xaml, only in F# code.
   - Lots of named, optional arguments, a fundamental tool for attribute-heavy DSLs in F#.

```
let view model dispatch =
    View.ContentPage(
        title = "Pocket Piggy Bank",
        content= View.Label(text = sprintf "Hello world!")
    )
```

THE VIEW DSL

1. Views are expressed using a DSL
2. It's like Xaml, only in F# code and simple

```
<?xml version="1.0" encoding="UTF-8"?>
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms" xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml" x:C
       <ContentPage.Content>
        <StackLayout Padding="20">
          <StackLayout Padding="20" VerticalOptions="Center">
            <Label Text="{Binding Path=[CounterValue], StringFormat=' value 1: {0}'}" HorizontalOptions="Center" Text
            <Label Text="{Binding Path=[CounterValue2], StringFormat=' value 2: {0}'}" HorizontalOptions="Center" Tex
            <Button Text="Increment" Command="{Binding Path=[IncrementCommand]}"   />
            <Button Text="Decrement" Command="{Binding Path=[DecrementCommand]}" />
            <Slider Maximum="10" Minimum="1" Value="{Binding Path=[StepValue]}" />
            <Label Text="{Binding Path=[StepValue], StringFormat='Step size: {0}'}" HorizontalOptions="Center" />
            <!-- This is included to show how the visibility of controls is controlled -->
        </StackLayout>
        <Button Text="Reset"  IsVisible="{Binding Path=[ResetVisible]}" HorizontalOptions="Center" Command="{Binding
      </StackLayout>
       </ContentPage.Content>
</ContentPage>
```

THE VIEW DSL

1. Views are expressed using a DSL
2. It's like Xaml, only in F# code and simple

```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = (model <> initModel () ))
    ]))
```

THE VIEW DSL

That original example

```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = model.Changed)
    ]))
```

THE VIEW DSL

That original example

```
let view (model: Model) dispatch =

  View.ContentPage(content=

    View.StackLayout(padding=30.0,verticalOptions = LayoutOptions.Center, children=[

        View.Label(text= string model.Count, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Increment", command= (fun () -> dispatch Increment))

        View.Button(text="Decrement", command= (fun () -> dispatch Decrement))

        View.Slider(minimum=0.0, maximum=10.0, value=model.Step,
             valueChanged= (fun args -> dispatch (SetStep (int args.NewValue))))

        View.Label(text=sprintf "Step size: %d" model.Step, horizontalOptions=LayoutOptions.Center)

        View.Button(text="Reset", horizontalOptions=LayoutOptions.Center,
             command=(fun () -> dispatch Reset), canExecute = not model.Unchanged)
    ]))
```

How easy is that?

FURTHER TOPICS

- Making it efficient
- Extensibility
- Maps, Charts, 2D Drawing, 3D Models etc.
- Live update tooling
- Animation
- Lots of View controls
- Phone integration (Notifications etc.)
- Practical considerations

MAKING IT EFFICIENT

```
let view model dispatch =
    …
    View.Slider(minimum=0.0, maximum=10.0,
        value=double model.Count,
        valueChanged=(fun args -> dispatch (SliderValueChanged args.NewValue)))
    …
```

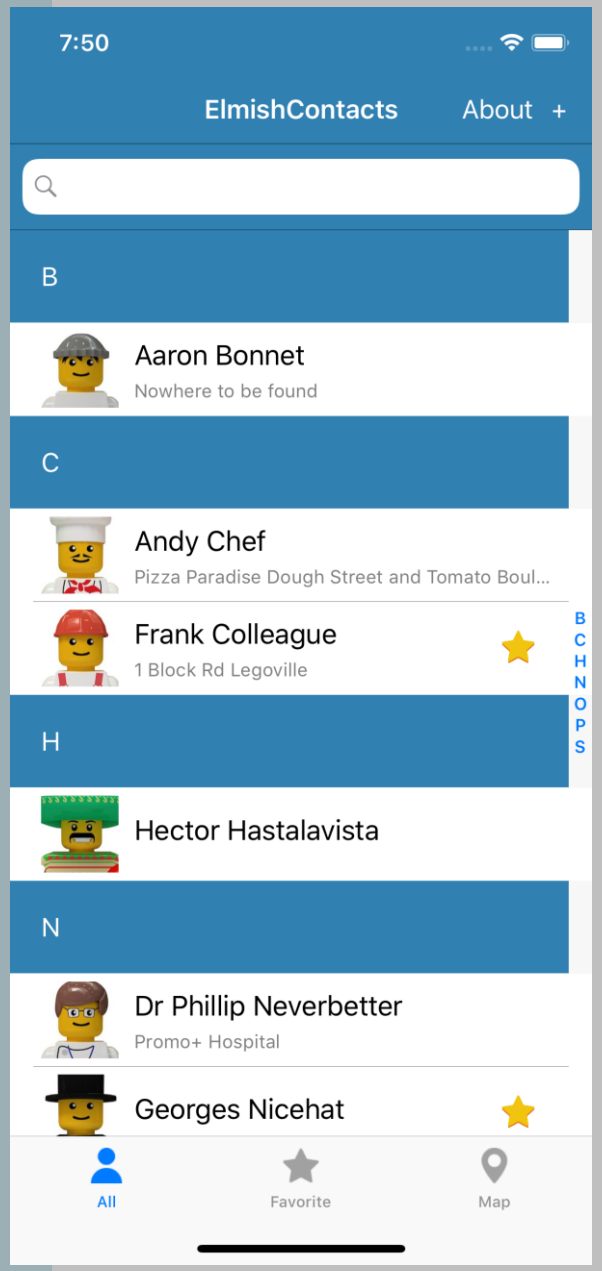
MAKING IT EFFICIENT

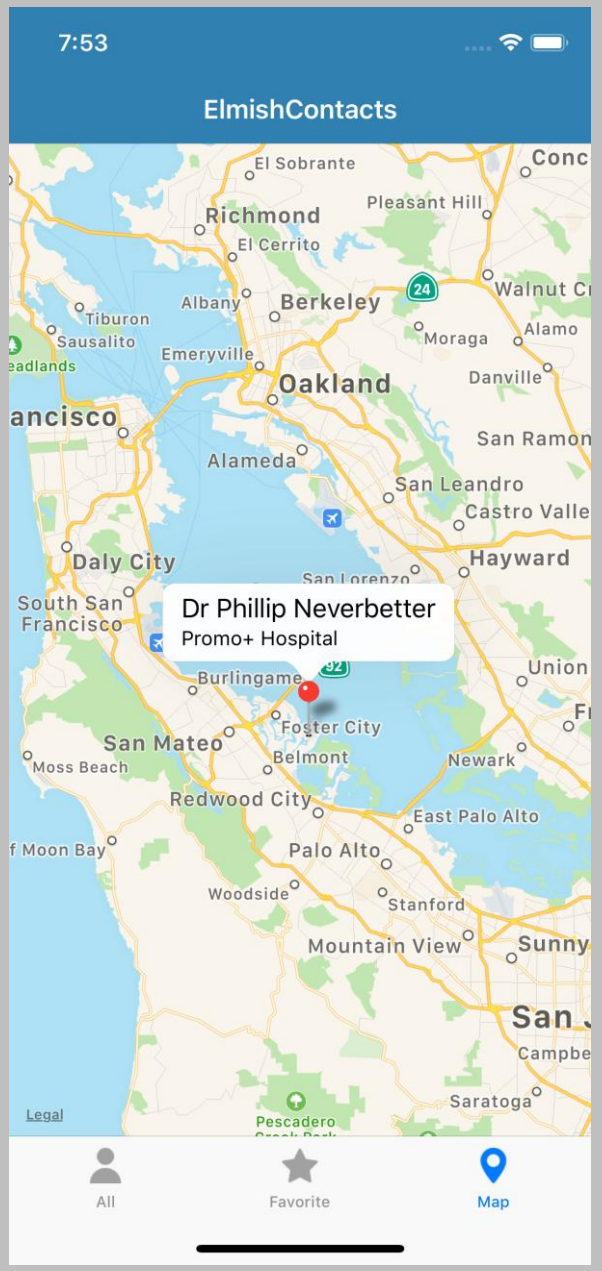
```
let view model dispatch =
    …
    dependsOn (…) (fun model (count, step) ->
        …
    )
    …
```

MAKING IT EFFICIENT

```
let view model dispatch =
    …
    dependsOn (model.Count, model.Step) (fun model (count, step) ->
        View.Slider(minimum=0.0, maximum=10.0,
            value=double count,
            valueChanged=(fun args -> dispatch (SliderValueChanged args.NewValue))))
    …
```

ELMISH CONTACTS





LEARN!

ElmishContacts is a great example

- On-device per-app SQL-Lite integration
- Using maps
- Asynchronous data loading onto maps
- Catching exceptions and being robust
- Sending SMS, making phone calls and Email using Xamarin.Essentials
- Multi-page with external messages reconciled via the composed app
- Editing and searching lists of things
- Taking photos
- Splash screens
- Decent design

LIVE UPDATE

- An experimental LiveUpdate mechanism is available
  - F# code for the core of your app is recompiled on host and interpreted on device
  - Communication to device via adb/http
  - Update times ~2sec
- Current limitations (please help!)
  - No state-migration as yet
  - Some manual start-up steps
  - DEBUG mode only
  - Some bugs/limitations in on-device interpreter
  - Some differences in semantics for on-device interpreter (e.g. typeof)

SUMMARY

Fabulous

=

Simple F#-friendly Elmish-style Functional Mobile App Programming

Please contribute and get involved!

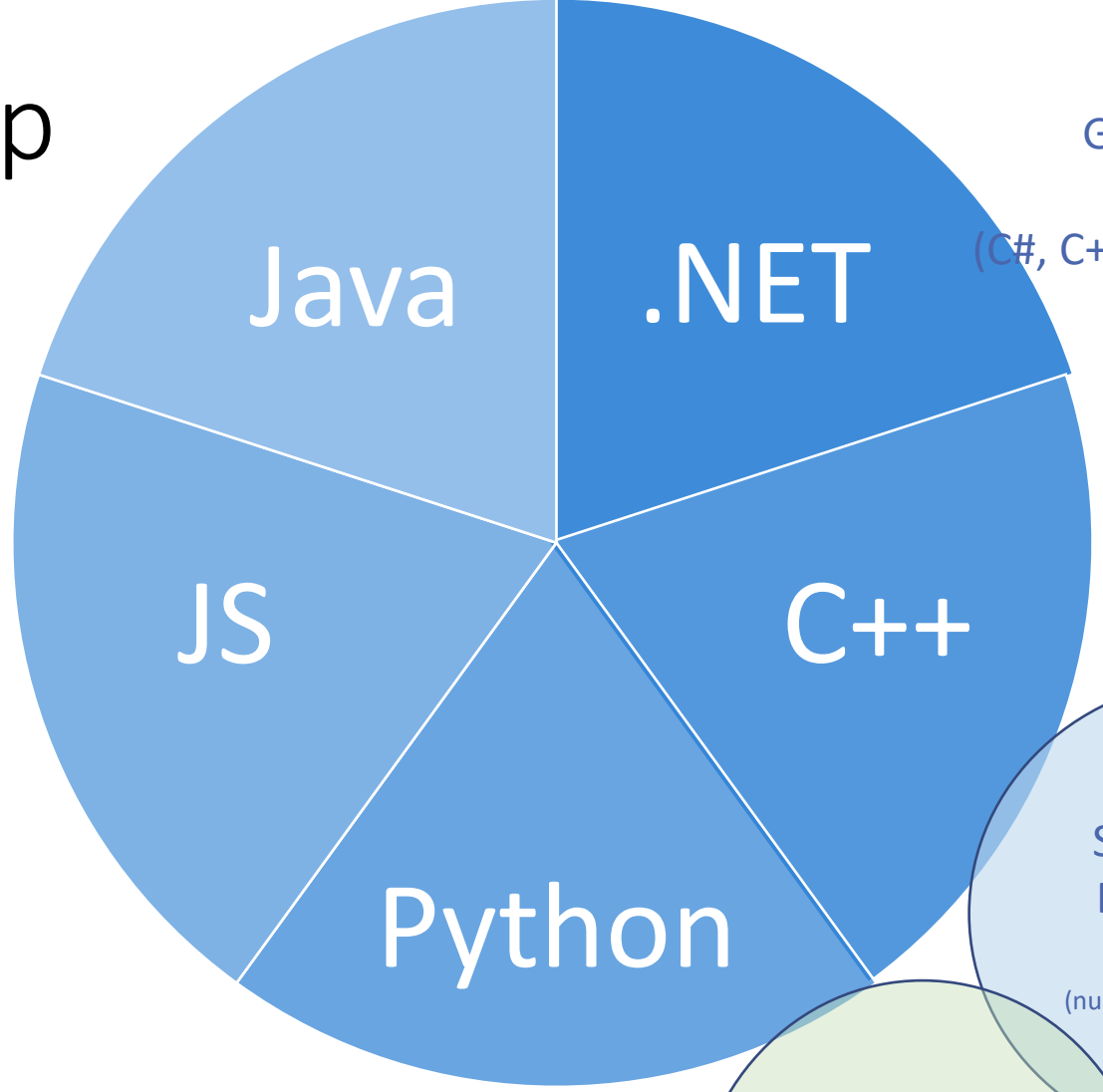
RESOURCES

- fsprojects.github.io/Fabulous
  - View snippets
  - Docs
  - Links to Samples
  - Templates
  - How to use LiveUpdate (for small apps)
- The AllControls sample
- Awesome-Fabulous
- Xamarin.Essentials
- Xamarin docs (e.g.Forms, Android, iOS setup and deploy)

PART II - APPLYING F# TO AI

Don Syme, Phillip Carter

Recap



General Purpose Development
(C#, C++, JS, Java, ½ Python)
5000

Statistics and Data Science
500
(numpy, pandas, matlab, R)

Data Scripting
500
(½ Python, ¼ bash, ¼ powershell)

Machine Learning
200
(TensorFlow, PyTorch, Keras, ½ numpy)

Systems Scripting
150
(¾ Powershell, ¾ bash)

Source:
#Stackoverflow tags

Situation

- Deep Learning is a critical, rapidly expanding part of the AI and ML landscape
- Deep Learning frameworks are programming languages (e.g. TensorFlow)
- There appears to be a “Gen 2.0” to “Gen 3.0” transition going on

    TF 1.0 → 2.0

    New host languages (e.g. Google Swift)

    Google Jax integrates tensors, gradients and control flow with Python code

    DL is expanding out from “static graphs”

    Many new ideas in DL architecture (e.g. Bayesian loss functions)

Problems

- Realistically, Gen 3.0 will be an evolution of Gen 2.0
  - There is no single revolution
  - Change is happening on many fronts
  - Language expressivity is a pain point for some advanced modelling
- Expert practitioners deeply wedded to current ecosystems (Python, TensorFlow, Torch)
  - Majority of creativity will happen in these ecosystems
- Practitioners experience real pain with Tensor programming
  - A lack of types makes Python Deep Learning (and data scripting) HARD and limits expressivity
  - Many, many shape mistakes
  - Incorrect striding
  - Subtle errors in missing layers, incorrect use of weights

Need

- Value can come in many areas
  - Better host languages (Swift, F#, Better Python…)
  - Better expressivity for Models…
  - Better checking of Models…
  - Better tooling for Models…
  - Better algorithms for Training….
  - Better cloud Hardware, Provisioning and Execution…

“F# for AI Models” - A sketch, not a reality

- A new take on authoring AI models in F#?
- A “configuration” of F#?
  - Fable and SAFE has shown how we can configure F# for a domain
  - Assume latest language features, default libraries
  - Assume ReflectedDefinition on all model code
- Deeply integrate gradient-based techniques
  - TensorFlow and Torch for real-world use
  - DiffSharp 1.0 as a more expressive fabric for auto-gradients?
  - Nested AD, recursive operations?
- Tooling for correct, robust AI Model Development?
- Make it fit with both .NET and Python (ML.NET, REPL, Jupyter Notebooks, Spark, …)

FM “v0.01” constructs

- Shape inference, checking and tooling
- Differentiable Tensors
  - Variables, Placeholders
  - Constants, Broadcasting, Slicing, Indexing
  - Standard math operators (+,*,-,/,abs,sin,cos,tan,sqr,sqrt…)
  - Reductions (mean,sum,prod,min,max,…)
  - Convolutions (2D, batched)
  - Gradients
  - diag, norm,trace, …
- V0.02 (tentative)
  - control flow
  - quotations
  - complete operations
  - Torch/DiffSharp backend

Results: Micro-example

```
#r "nuget: FSharp.AI.Models, Version=0.1"
```

```
// A numeric function of two parameters, returning a scalar, see
// https://en.wikipedia.org/wiki/Gradient_descent
let f (xs: Vector) : Scalar =
    sin (0.5 * sqr xs.[0] - 0.25 * sqr xs.[1] + 3) * -cos (2 * xs.[0] + 1 - exp xs.[1])
```

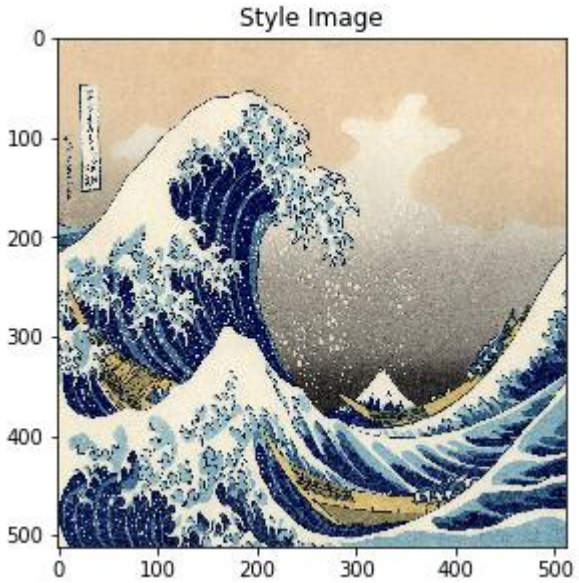
```
// Pass this to gradient descent
let train numSteps = GradientDescent.train f (vec [ -0.3; 0.3 ]) numSteps
```

```
let results = train 200 |> Seq.last
```

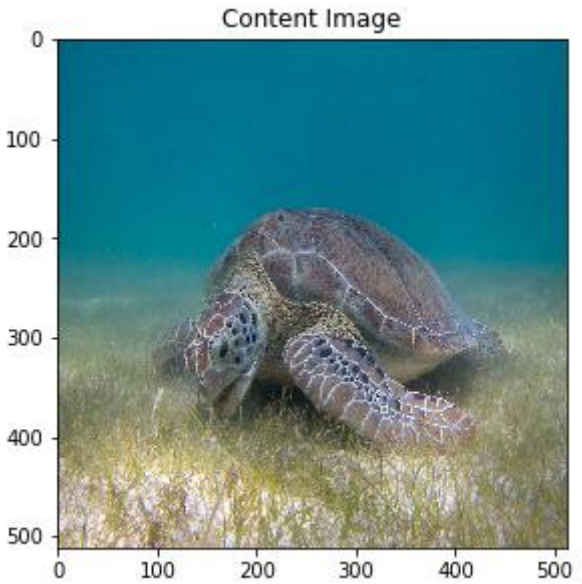
Results: Macro-example

- Explaining and Neural Style Transfer

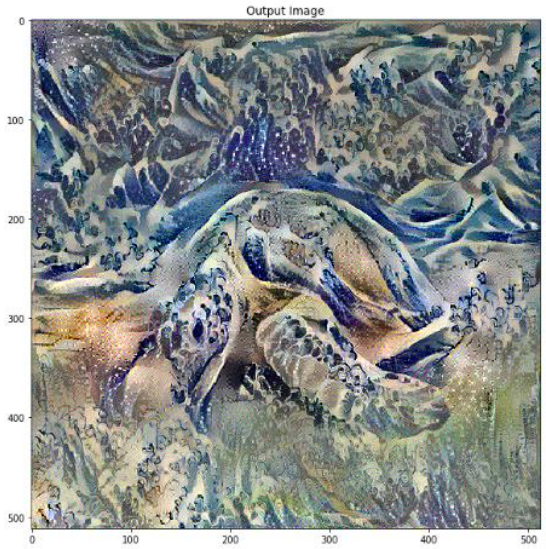
- Demo



Train a DNN



Use the DNN



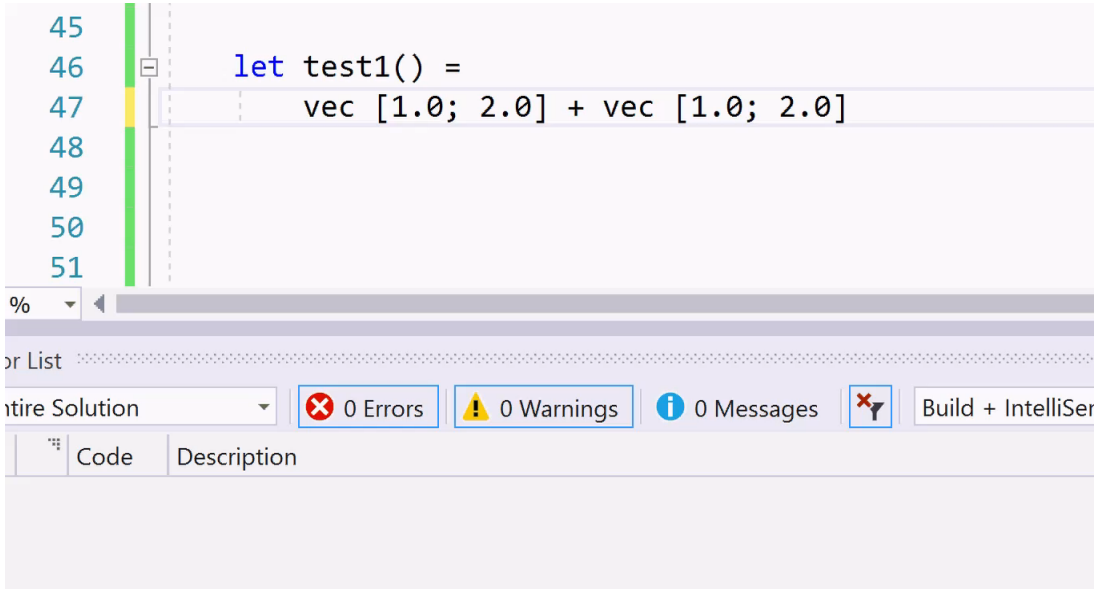
Tooling for Tensor Shapes

- One holy grail of DNN programming tooling is **tensor shape checking**
  - Tensors are multi-dimensional arrays used to represent source and emergent data
  - Tensors have **shapes** like “10 x 320 x 720 x 4”
  - Logical these shapes are **symbolic** like “N x H x W x C”
- Your choices:
  - No checking = lots of bugs
  - Use type system = extreme complexity
- A new choice!
  - **Use live checking = live testing + checking collecting information for tooling**

Tooling for Tensor Shapes: demo

- Explaining and Using Deep Learning

- Finding a shape bug



Tooling for Tensor Shapes: demo

- Explaining and Using Deep Learning

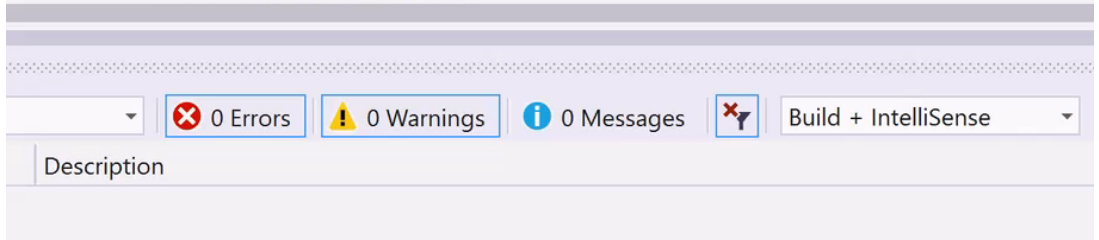
- Finding a shape bug

```
let residual_block (filter_size, name) input =
    let tmp = conv_layer (128, filter_size, 1, true, name
    input + conv_layer (128, filter_size, 1, false, name

let to_pixel_value (input: DT<double>) =
    tanh input * v 150.0 + (v 255.0 / v 2.0)

let clip min max x =
   DT.ClipByValue (x, v min, v max)

// The style-transfer network.
let PretrainedFFStyleVGG images =
    images
    |> conv_layer (32, 9, 1, true, "conv1")
    |> conv_layer (64, 3, 2, true, "conv2")
    |> conv_layer (128, 3, 2, true, "conv3")
    |> residual_block (3, "resid1")
    |> residual_block (3, "resid2")
    |> residual_block (3, "resid3")
    |> residual_block (3, "resid4")
    |> residual_block (3, "resid5")
    |> conv_transpose_layer (64, 3, 2, "conv_t1")
    |> conv_transpose_layer (32, 3, 2, "conv_t2")
    |> conv_layer (3, 9, 1, false, "conv_t3")
    |> to_pixel_value
    |> clip 0.0 255.0
```



Tooling for Tensor Shapes

- Results
  - Static TensorShape checking alone gets the interest of DNN experts.
  - With this we are already world-leading in this area
  - Anecdotal: helps explain Deep Learning to 13 yr olds (my son)
  - It’s applied to Gen 2.0 models, but will work just as well with Gen 3.0

- Aside: The LiveChecking technique has many potential applications
  - It’s not full types, but it doesn’t matter – it’s statically checked and tooled
  - Checking ML.NET schema in pipeline composition
  - Checking other DSLs

Next Steps

1. This was experimental investigation, it’s my job.

2. This is not remotely ready for production, I’m just having investigating

3. Don’t let this disrupt your world view!

4. To watch: SciSharp. We will switch to their TensorFlow.NET

IMPROVING F#

Don Syme, Phillip Carter

F# 4.6

- Currently in Preview and ships with VS 2019 GA
  - And corresponding .NET Core 2.1 and 2.2 updates
- Anonymous Records
  - Supported in Fable too!
- FSharp.Core improvements
  - ValueOption function parity

F# 5.0 – NULLABLE REFERENCE TYPES

F# has little problem with nulls. C# has a massive problem.

Today:

- F# types do not have null as a normal value.
- .NET types do, but it is rare to encounter null values flowing out of .NET libraries.

C# is trying to make progress on this problem by adding "use-site nullness annotations"

.NET libraries will have nullness annotations. We can make use of this

F# 5.0 – NULLABLE REFERENCE TYPES

Design philosophy

1. Design for F# first. Make sure it "looks right" as an F# language feature

2. Keep F# simple. The feature will be rarely used in F#. You will only use this feature when using .NET libraries and "null".

3. This is a warning-level informational feature. It is not a strong "soundness" feature, but a helpful aid.

4. `option`/`Some`/`None` remains the normal way to represent missing information in F#.

5. Warnings only activated for new projects (`/langversion:5.0`)

F# 5.0 – NULLABLE REFERENCE TYPES

Constructs:

Types:

```
string?    - has null as a normal value

string     - does not have null as a normal value
```

Introduction:

```
let s1 : string = null   // warning
let s2 : string? = if today then "abc" else null // ok
```

Elimination:

```
match s with
| NonNull v -> …
| null -> …
```

F# 5.0 – NULLABLE REFERENCE TYPES

Refinements, see RFC

1. Library functions `withNull`, `isNull`, `nonNull` and more
2. "Ambivalence" for types coming from non-annotated .NET assemblies
3. "obj" is always ambivalent
4. Possible simpler ways to eliminate

F# 5.0 – OTHER THINGS

Possible items:

1. FS-1070: Relax indentation warnings
2. FS-1043: Extension methods visible to trait constraints
3. FS-1069: Implict yields (for Fable and Fabulous)
4. FS-1068: Open static classes (for Fable, Fabulous and Spark)
5. FS-1003: nameof operator
6. FS-1071: Witness passing info in quotations (for Fable and "F# for AI Models")
7. FS-1063: Applicatives syntax in computation expressions "let! … and! …"
8. FS-1056: Overloads of custom operators in computation expressions

DELIVERING F#

A Chat with Phillip Carter

DELIVERING F#

Topics:

1. Delivery in .NET SDK 2.1, 2.2

2. FSharp.Compiler.Service and Fable up-to-date and community delivered. Mono lagging a little.

3. Towards Visual Studio 2019

4. Towards .NET SDK 3.0